Project Step 6

Group 46: Joshua Cicchinelli, Mauricio Marin Gutierrez, and Son Thai Do Nguyen

URL Link

http://classwork.engr.oregonstate.edu:1995

Executive Summary

Our library management system evolved significantly through iterative feedback and fast development, transforming from a basic outline into a fully functional web application.
Initial Design & Normalization: We began by creating a database outline or "blueprint" then revised it to achieve normalization while drafting the DDL with sample data. Peer feedback from Skyla Chen and Jonathan Salfen during Step 1 revealed that our original schema lacked proper M:N relationships; we had initially linked Books directly to Authors, which would have caused data redundancy. Based on this, we added junction tables BookAuthors and BookGenres, eliminating duplication. Additional refinements included adding a bookMedia attribute, standardizing phone number lengths, and ensuring naming consistency throughout.
UI Development & Collaboration: We adapted the CS340 lecture code to build our website's foundation, customizing it with our own schema and tables. Creating a Git repository greatly aided team coordination, allowing us to track changes and merge work seamlessly. Cameron Brooks identified inconsistencies in our phone number field (VARCHAR(10) in our outline vs VARCHAR(12) in our DDL), which we standardized to VARCHAR(12) to accommodate formatted numbers. Thai-Binh Guerreiro suggested using ENUM for bookMedia instead of freeform VARCHAR, which we implemented to ensure data consistency.
CRUD Implementation: We first created a reset procedure and implemented basic CRUD operations while applying a stylesheet to make the UI clean and readable. Alexander Jones and Brandon Pacewic noted that our initial interface displayed raw foreign key IDs instead of readable labels, so we redesigned all tables to show book titles, author names, and patron names using JOIN queries, making the interface intuitive for non-technical users. Based on feedback from Cody Strehlow and Zachary Sherman about missing DELETE functionality, we expanded to full CRUD operations for all tables, including intersection tables, and created stored procedures for all CUD operations. Delving into route handling and stored procedure creation was critical during this phase.
Final Polish & Testing: We completed the project by addressing remaining bugs, which required thorough testing of edge cases and database constraints. The very last step was compiling this summary and packing all deliverable into a zip file for submission.
Reflection on AI Tools: Throughout development, we used ChatGPT strategically, it helped debug complex JOIN queries in our Books page and assisted with syntax errors in stored procedures. However, AI-generated code required manual verification. We learned that AI excels at specific task like debugging and boilerplate generation, but architectural decisions require human oversight.

Project Outline and Database Outline, ERD Schema, Sample Data, & Screen Captures Updated Version

Overview:

This library database will keep an inventory of all the books that a medium size library holds, up to 43,000 books. It will be able to serve 15,000-25,000 registered members,with a centralized database. Furthermore, this system will store data on each of the books as well as facilitate a record of transactions that patrons will be able to perform when they borrow books. Clients will be able to use this software at the same time; 24/7. Without this, the library would have to rely on fragmented or manual tools.

Database Outline:

- **Books-** Records the details of books in the library
    - bookID: INT, auto_increment, unique, not NULL, PK
    - ISBN: VARCHAR(13), not NULL, UQ
    - title: VARCHAR(255), not NULL
    - publicationYear: YEAR
    - publisher: VARCHAR(100)
    - bookMedia: ENUM('Hardcover','Paperback','E-Book','Audio'), not NULL

    - Relationships:
        - A M:N relationship between Books and Authors is implemented using BookAuthors intersection table.
        - A M:N relationship between Books and Genres is implemented using the BookGenres intersection table.
        - A 1:M relationship between Books and BookCopies is implemented with bookID as a FK inside of BookCopies

- **Patrons-** Records library members who can borrow materials.
    - patronID: INT, auto_increment, unique, not NULL, PK
    - libraryCardNumber: VARCHAR(20), not NULL, UQ
    - firstName: VARCHAR (50), not NULL
    - lastName: VARCHAR (50), not NULL
    - email: VARCHAR (100), not NULL, UQ
    - phone: VARCHAR (12)

    - Relationship:
        - A 1:M relationship between Patrons and Loans is implemented with patronID as a FK inside of Loans.

- **Loans-** Tracks all book checkout transactions between patrons and book copies, recording dates, fees and status. Each row is a separate transaction showing what happens over time.

    - loanID: INT, auto_increment, unique, not NULL, PK
    - copyID: INT, not NULL, FK
    - patronID: INT, not NULL, FK
    - checkoutDate: DATE, not NULL, default (CURRENT_DATE)
    - dueDate: DATE, not NULL
    - returnDate: DATE, NULL
    - lateFee: DECIMAL (5,2), default 0.00
    - status: ENUM ('Active', 'Returned', 'Overdue', 'Lost'), default 'Active'

    - Relationships:
        - A 1:M relationship between Patrons and Loans is implemented with patronID as a FK inside of Loans.
        - A 1:M relationship between BookCopies and Loans is implemented with copyID as FK inside of Loans.

- **Authors-** Stores information about authors of the book(s)
    - authorID: INT, auto_increment, unique, not NULL, PK
    - firstName: VARCHAR(50), not NULL
    - lastName: VARCHAR(50), not NULL
    - birthYear: SMALLINT, NULL

    - Relationship(s):
        - A M:N relationship between Books and Authors is implemented using the BookAuthors intersection table.

- **Genres-** Stores a list of genres that can be associated with books
    - genreID: INT, auto_increment, unique, not NULL, PK
    - genreName: VARCHAR(50), unique, not NULL

    - Relationship(s):
        - A M:N relationship between Books and Genres is implemented using the BookGenres intersection table.

- **BookCopies-** Track the physical copy of a book (Some book titles can have multiple copies)
    - copyID: INT, auto_increment, unique, not NULL, PK
    - bookID: INT, not NULL, FK
    - acquisitionDate: DATE, not NULL
    - condition: VARCHAR(20), not NULL
    - location: VARCHAR(50), not NULL
    - status: VARCHAR(20), not NULL

    - Relationship(s):

        - A 1:M relationship between Books and BookCopies is implemented with book ID as a FK inside of BookCopies

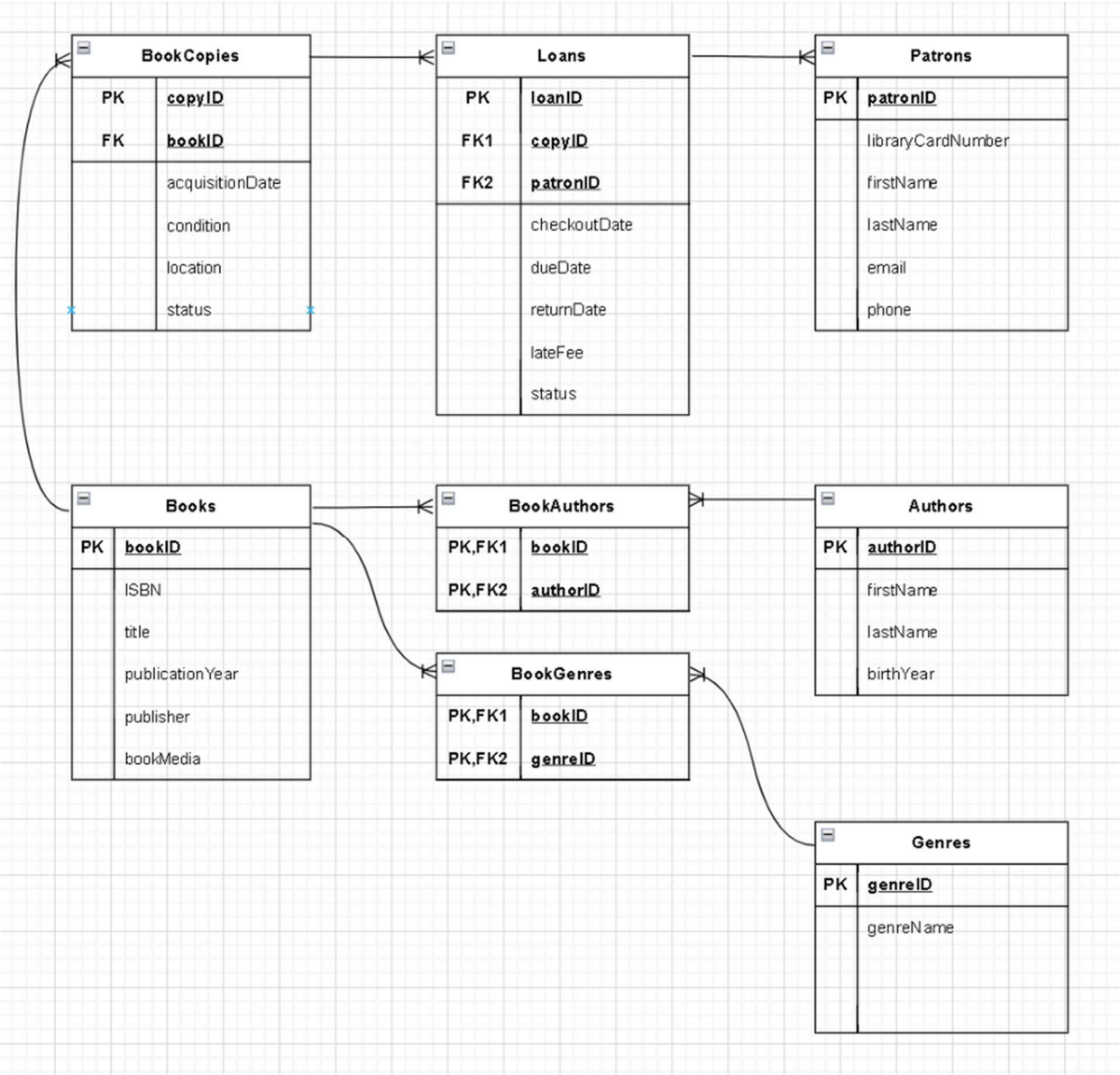
- **BookAuthors-** A join table to associate books and authors
    - bookID: INT not NULL, PK, FK
    - authorID: INT not NULL, PK, FK

    - Relationship(s): A M:N relationship between Books and Authors, where a book may have multiple authors and an author may have written multiple books.

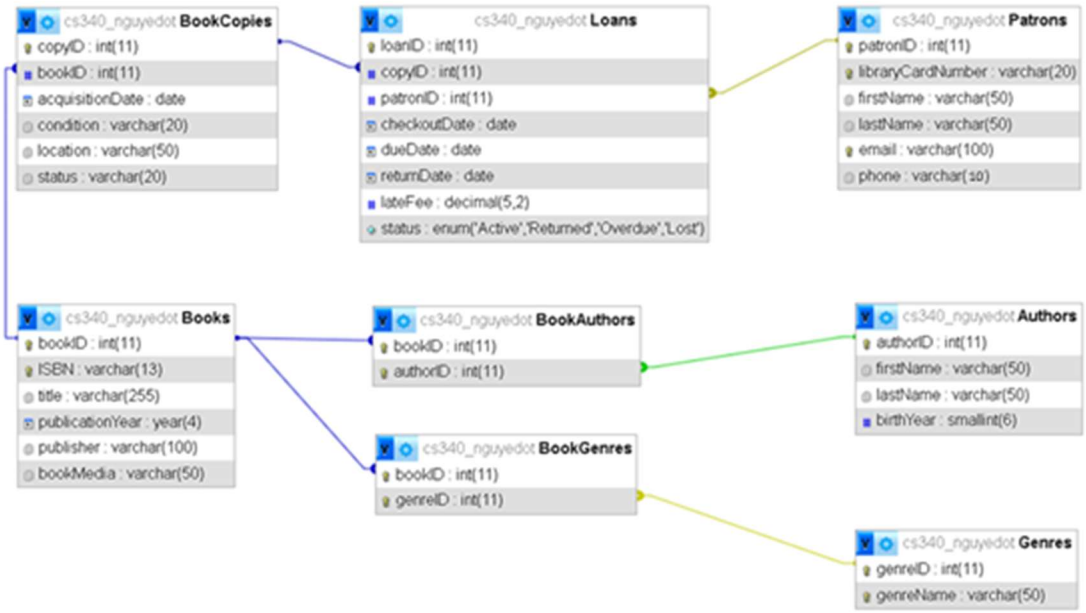
- **BookGenres-** A join table to associate genres and books
    - bookID: INT not NULL, PK, FK
    - genreID: INT not NULL, PK, FK

    - Relationship(s): A M:N relationship between Books and Genres, where a book may have multiple genres and a genre may have multiple books as part of it.

Entity Relationship Diagram:



Schema Diagram:



Sample Data:

Books

←T→	bookID	ISBN	title	publicationYear	publisher	bookMedia
Edit Copy Delete	1	9780439708180	Harry Potter and the Sorcerer's Stone	1997	Scholastic	Physical
Edit Copy Delete	2	9780439064873	Harry Potter and the Chamber of Secrets	1998	Scholastic	Physical
Edit Copy Delete	3	9780307743657	The Shining	1977	Doubleday	Physical

Patrons

←T→	patronID	libraryCardNumber	firstName	lastName	email	phone
Edit Copy Delete	1	LC1001	Mauricio	Gutierrez	mauricio.gutierrez@email.com	5415552001
Edit Copy Delete	2	LC1002	Joshua	Cicchinelli	joshua.cicchinelli@email.com	5415552002
Edit Copy Delete	3	LC1003	Son	Nguyen	son.nguyen@email.com	*NULL*

Authors

←T→	authorID	firstName	lastName	birthYear
Edit Copy Delete	1	J.K.	Rowling	1965
Edit Copy Delete	2	Stephen	King	1947
Edit Copy Delete	3	H.P.	Lovecraft	1890

Genres

←T→	genreID	genreName
Edit Copy Delete	1	Fantasy
Edit Copy Delete	3	Fiction
Edit Copy Delete	2	Horror

BookCopies

←T→	copyID	bookID	acquisitionDate	condition	location	status
Edit Copy Delete	1	1	2023-03-01	Good	Shelf F1	Available
Edit Copy Delete	2	1	2023-05-12	Worn	Shelf F1	Checked Out
Edit Copy Delete	3	2	2022-10-10	Good	Shelf F2	Available
Edit Copy Delete	4	3	2021-09-20	Good	Shelf H1	Available

BookAuthors

				bookID	authorID
	Edit	Copy	Delete	1	1
	Edit	Copy	Delete	2	1
	Edit	Copy	Delete	3	2

BookGenres

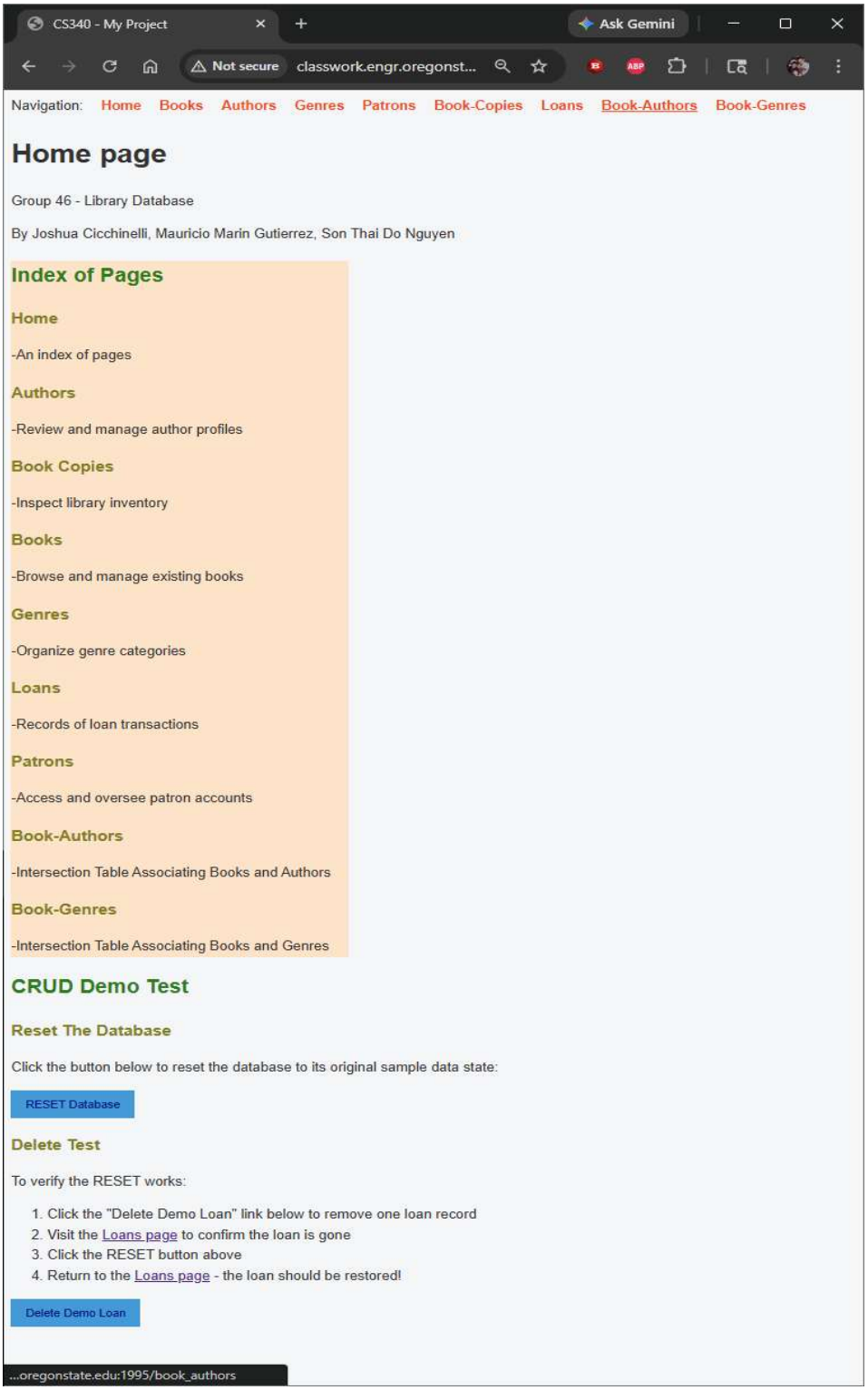
				bookID	genreID
	Edit	Copy	Delete	1	1
	Edit	Copy	Delete	2	1
	Edit	Copy	Delete	3	2

Loans

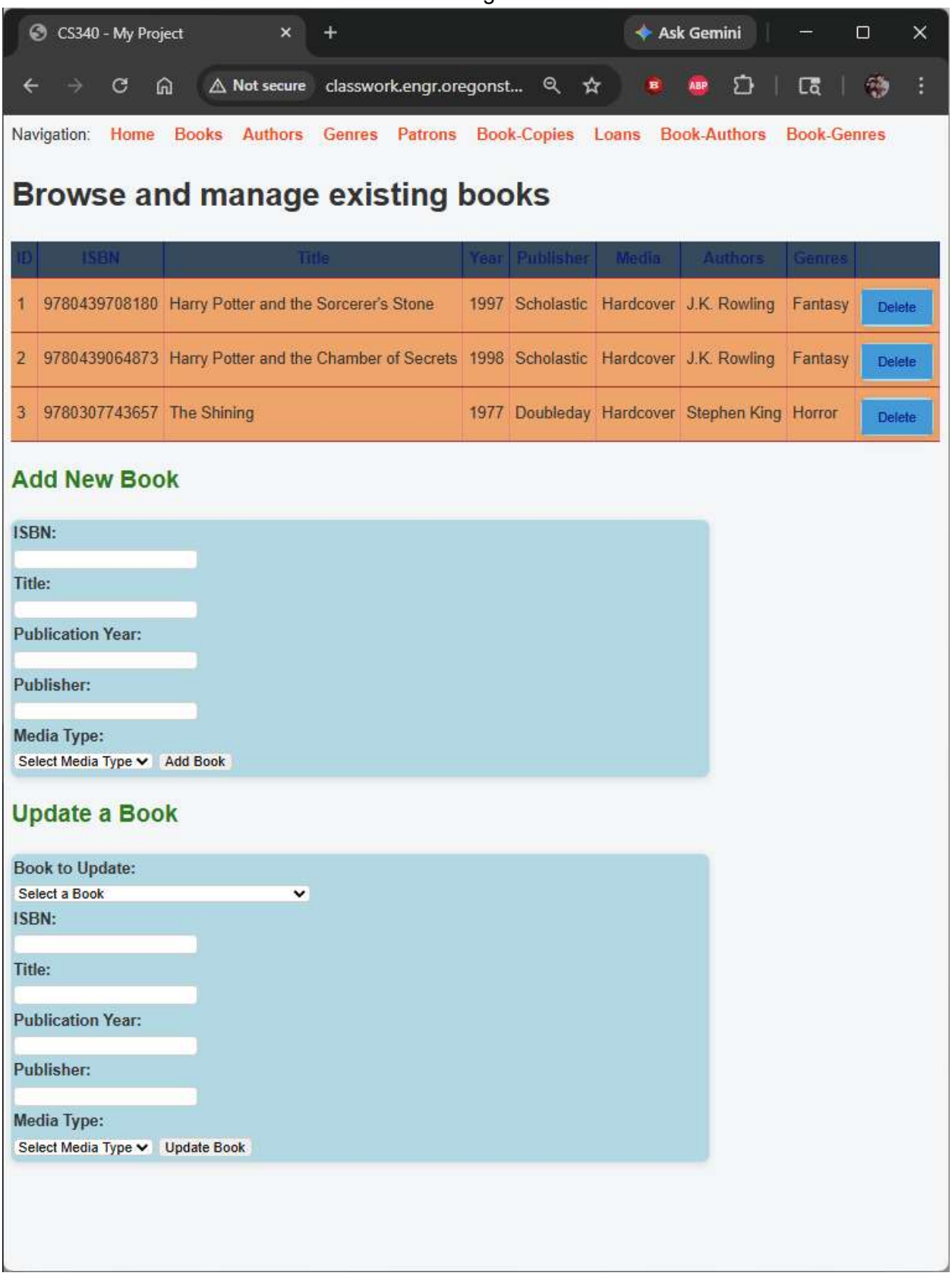
				loanID	copyID	patronID	checkoutDate	dueDate	returnDate	lateFee	status
	Edit	Copy	Delete	1	2	1	2026-02-01	2026-02-15	*NULL*	0.00	Active
	Edit	Copy	Delete	2	1	2	2025-11-01	2025-11-15	2025-11-14	0.00	Returned
	Edit	Copy	Delete	3	4	3	2025-10-10	2025-10-24	2025-10-30	1.50	Overdue

Screen Captures:

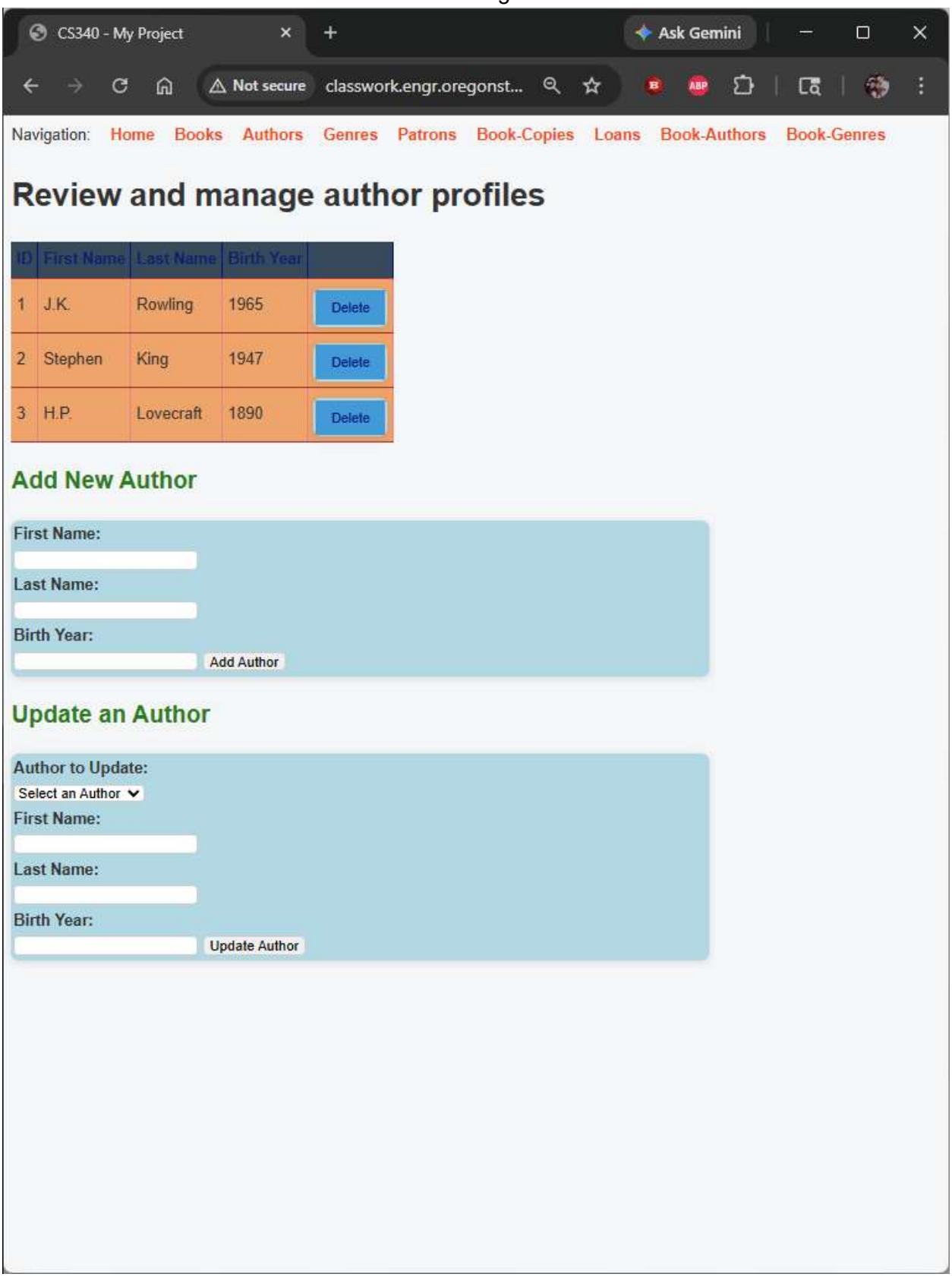
DISPLAY/DELETE & RESET DB - Home Index Page



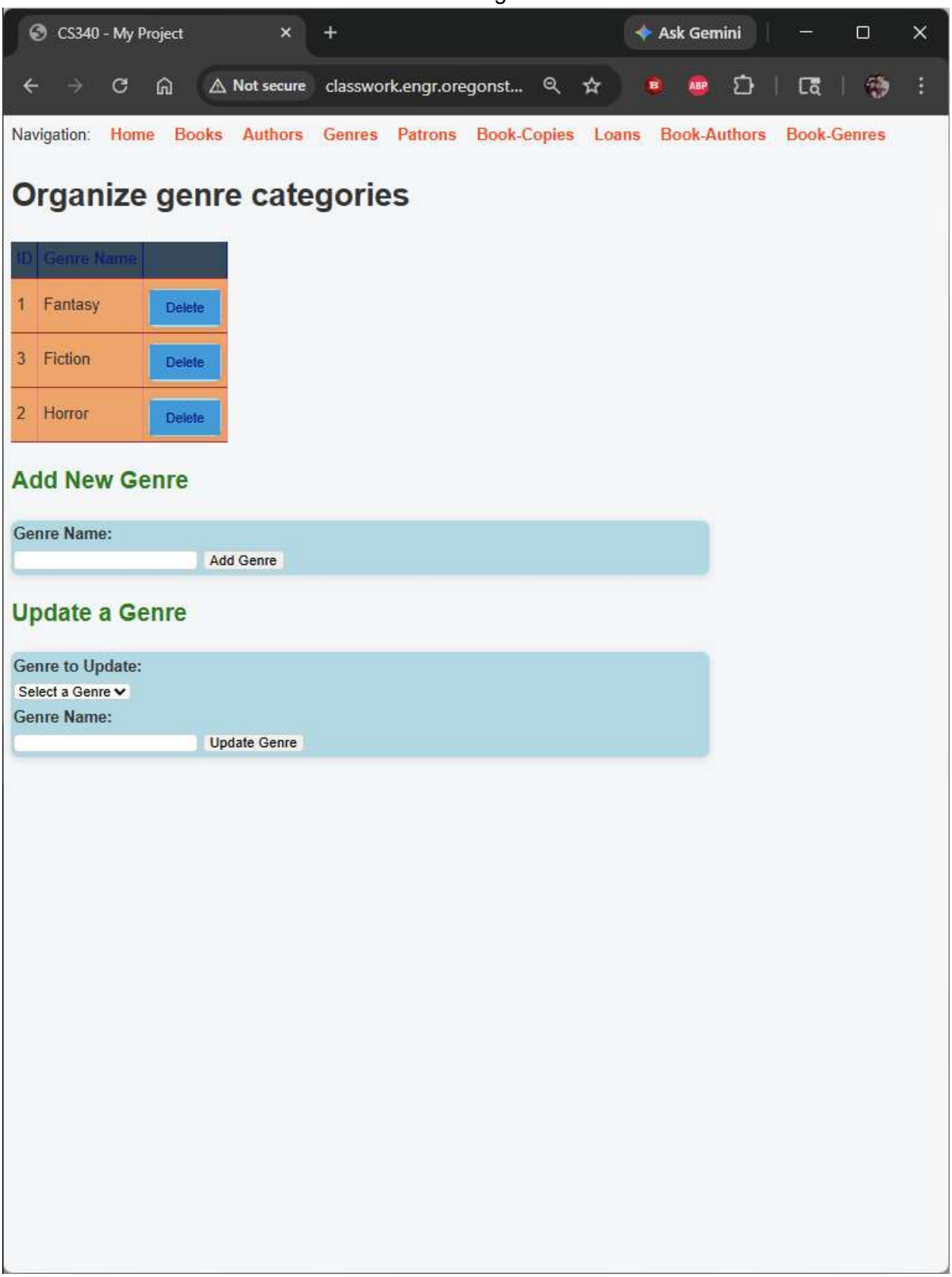
DISPLAY/CREATE/UPDATE/DELETE Books Page



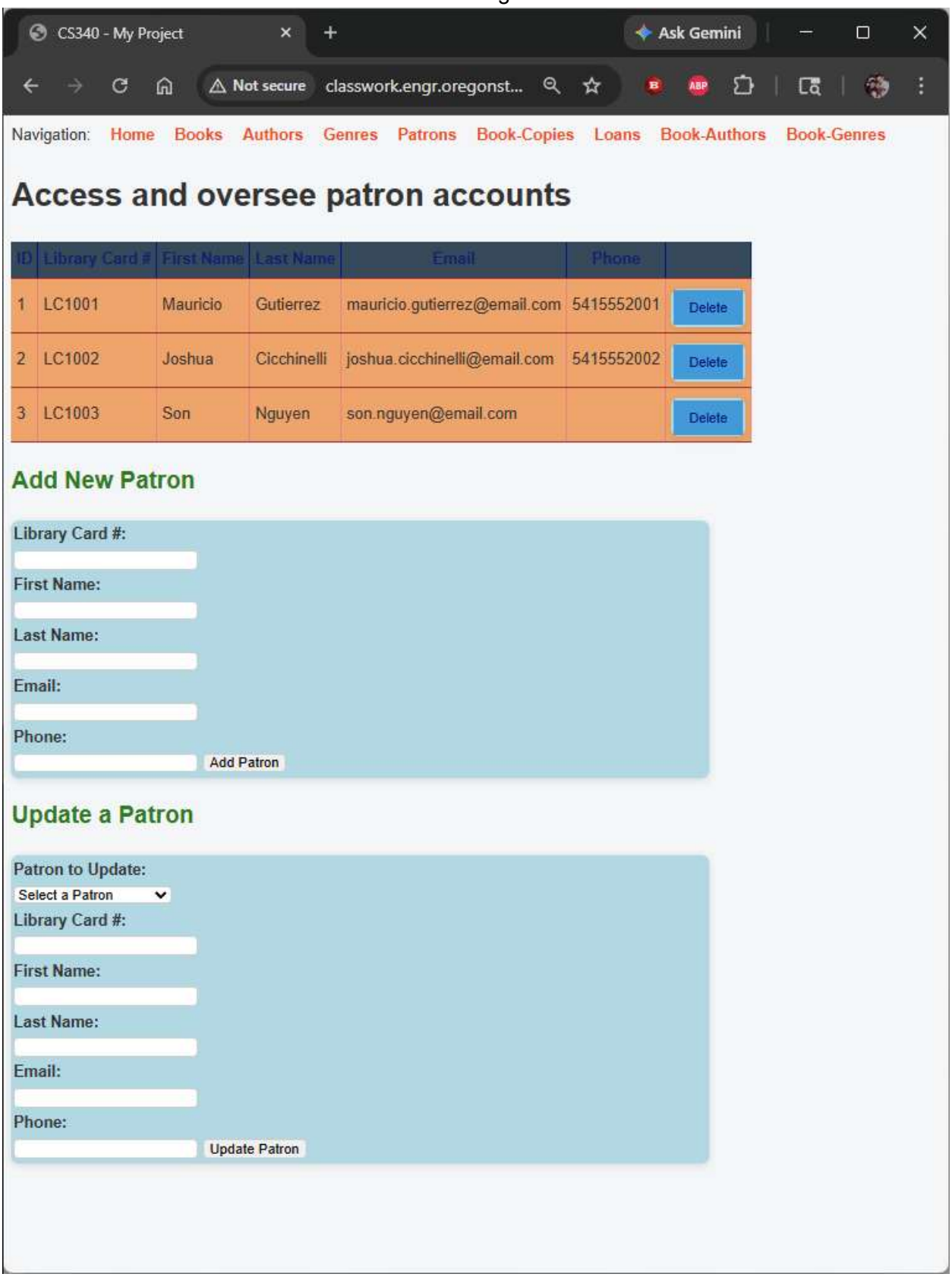
DISPLAY/CREATE/UPDATE/DELETE Authors Page



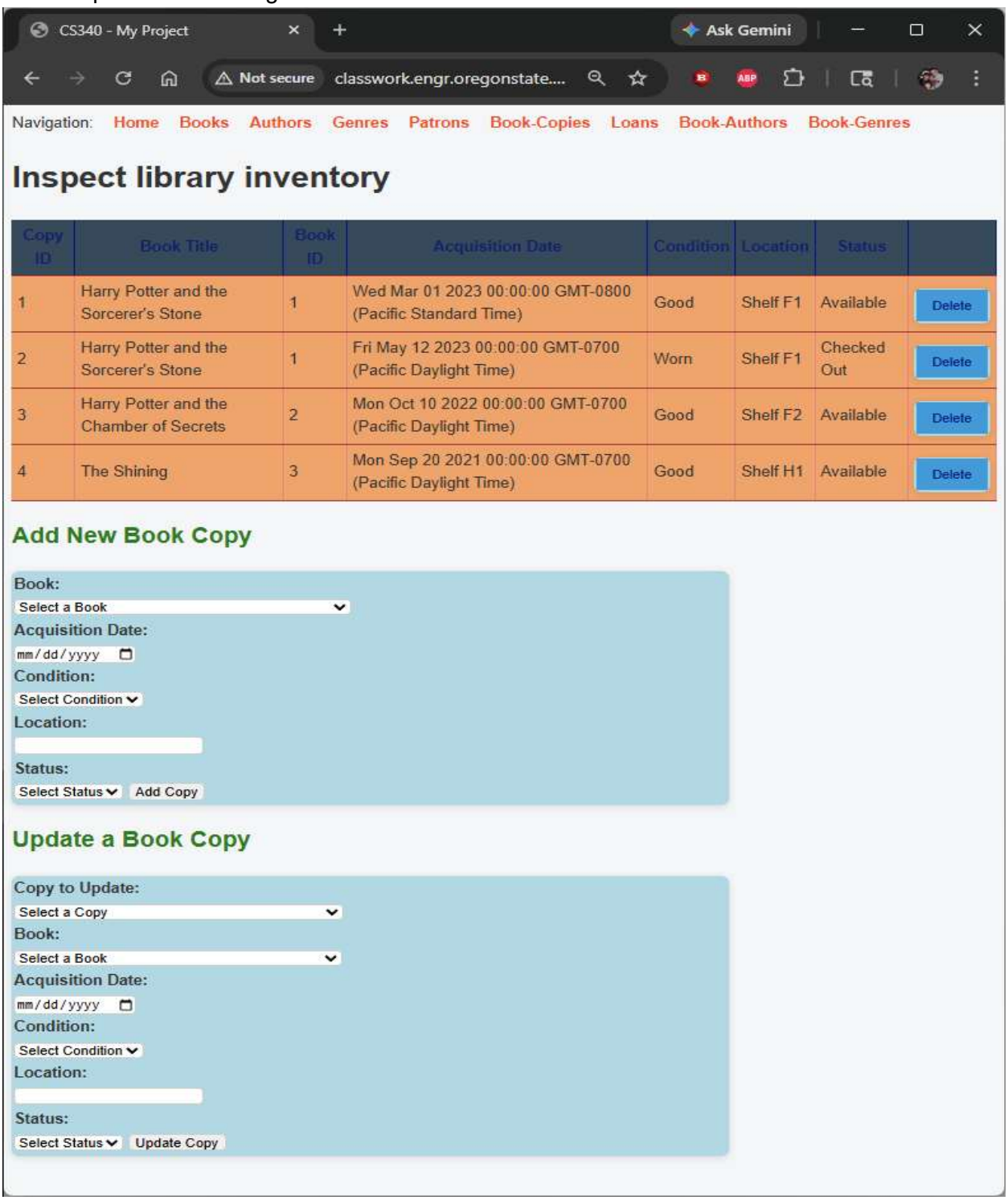
DISPLAY/CREATE/UPDATE/DELETE Genres Page



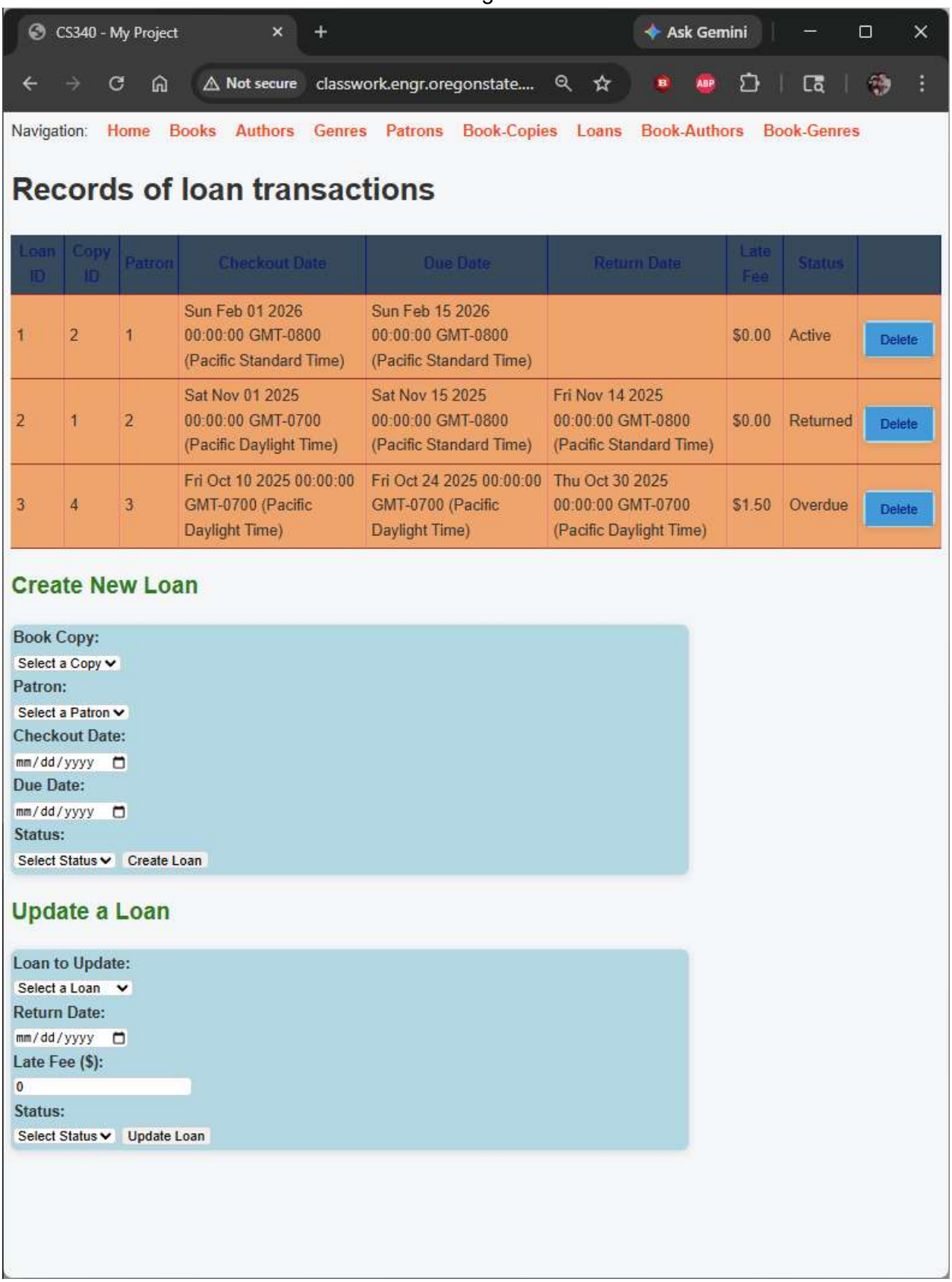
DISPLAY/CREATE/UPDATE/DELETE Patrons Page



DISPLAY/CREATE/UPDATE/DELETE & Delete from a M:N & Update of a M:N
Book Copies Junction Page



DISPLAY/CREATE/UPDATE/DELETE Loans Page



CS340 - My Project

Ask Gemini

Not secure classwork.engr.oregonstate....

Navigation: Home Books Authors Genres Patrons Book-Copies Loans Book-Authors Book-Genres

Records of loan transactions

Loan ID	Copy ID	Patron	Checkout Date	Due Date	Return Date	Late Fee	Status	
1	2	1	Sun Feb 01 2026 00:00:00 GMT-0800 (Pacific Standard Time)	Sun Feb 15 2026 00:00:00 GMT-0800 (Pacific Standard Time)		$0.00	Active	Delete
2	1	2	Sat Nov 01 2025 00:00:00 GMT-0700 (Pacific Daylight Time)	Sat Nov 15 2025 00:00:00 GMT-0800 (Pacific Standard Time)	Fri Nov 14 2025 00:00:00 GMT-0800 (Pacific Standard Time)	$0.00	Returned	Delete
3	4	3	Fri Oct 10 2025 00:00:00 GMT-0700 (Pacific Daylight Time)	Fri Oct 24 2025 00:00:00 GMT-0700 (Pacific Daylight Time)	Thu Oct 30 2025 00:00:00 GMT-0700 (Pacific Daylight Time)	$1.50	Overdue	Delete

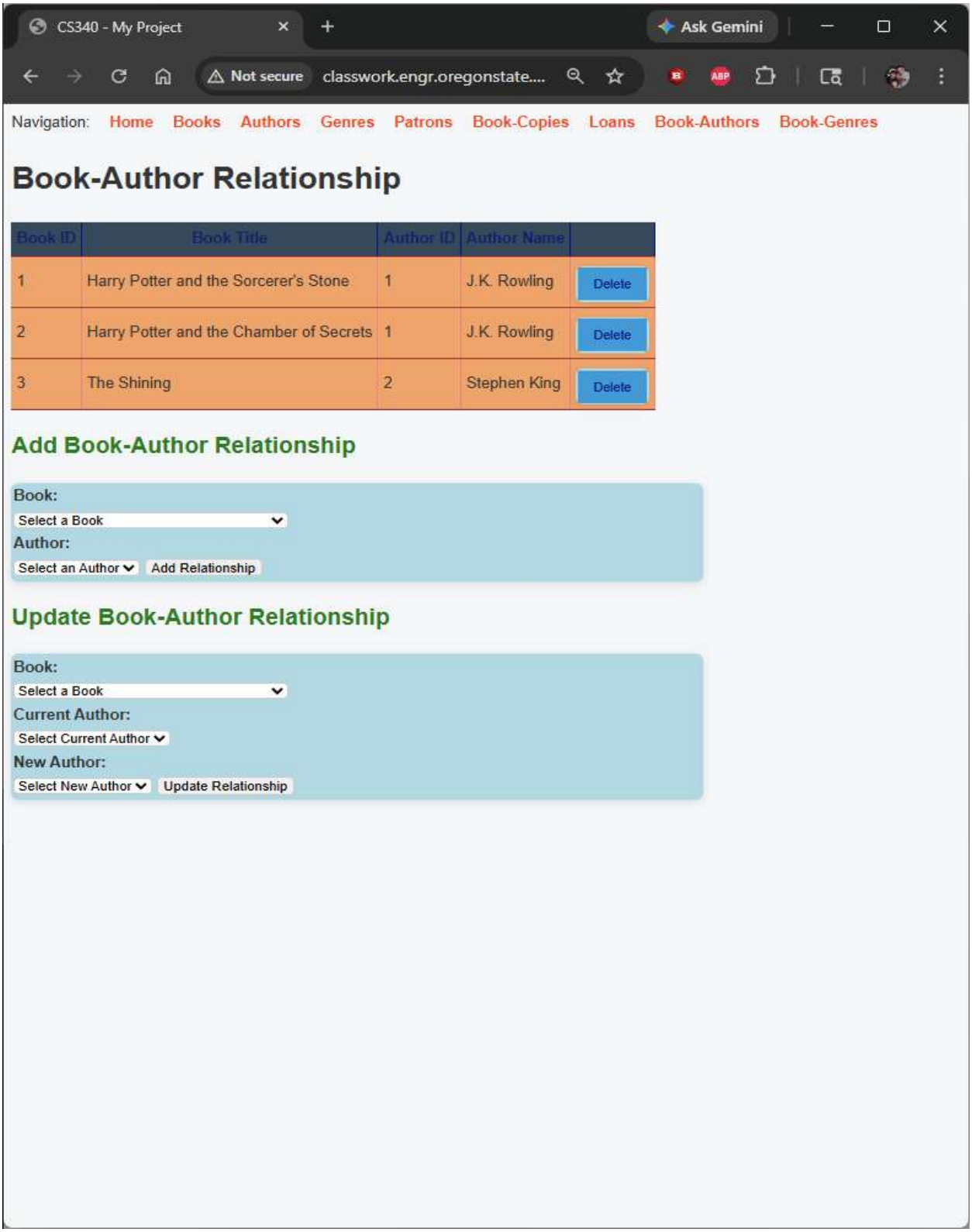
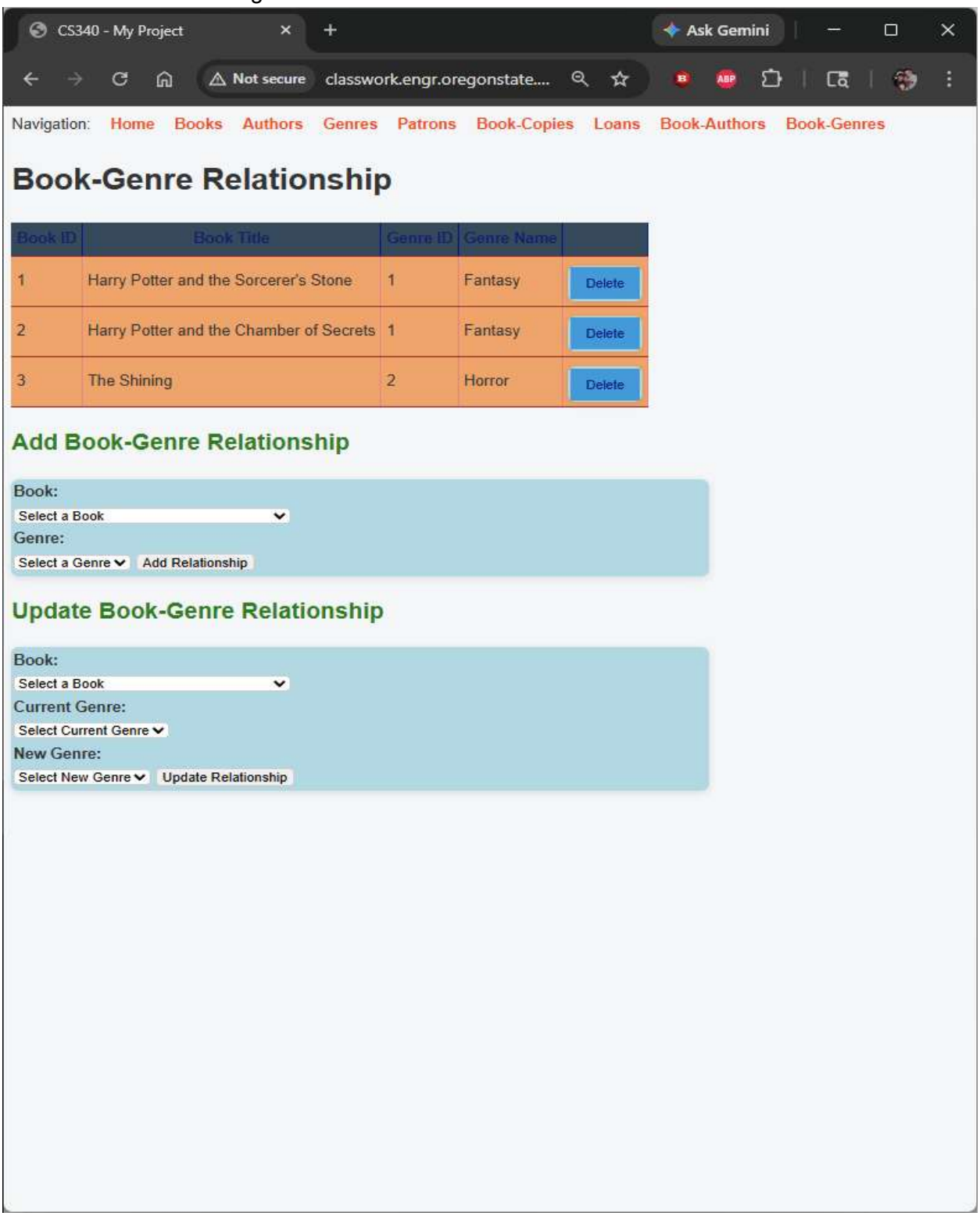
Create New Loan

Book Copy:
Select a Copy
Patron:
Select a Patron
Checkout Date:
mm/dd/yyyy
Due Date:
mm/dd/yyyy
Status:
Select Status Create Loan

Update a Loan

Loan to Update:
Select a Loan
Return Date:
mm/dd/yyyy
Late Fee ($):
0
Status:
Select Status Update Loan

DISPLAY/CREATE/UPDATE/DELETE & Delete from a M:N & Update of a M:N
Book Author Junction Page



DISPLAY/CREATE/UPDATE/DELETE & Delete from a M:N & Update of a M:N
Book Genre Junction Page